

Systèmes d'exploitation I

Chapitre 1 : Introduction & Généralités

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

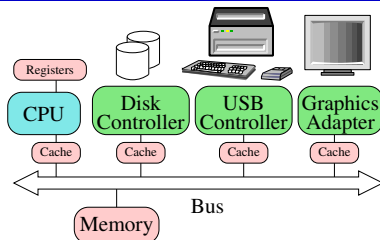
Sommaire

- 1 Introduction
- 2 OS : rôle, évolution
- 3 UNIX
- 4 Organisation en couches et notion d'interfaces
- 5 Conclusion

Système informatique

Le matériel

- CPU (Central Processing Unit)
- Structures de stockage
- Périphériques d'E/S
- Communication via un **bus** :



- **Bus de données**: véhicule les données
- **Bus d'adresse**: véhicule des adresses et conditionne la taille mémoire adressable
- **Bus de commande**: véhicule tous les signaux utilisés pour synchroniser les différentes activités : signaux d'horloge, R/W, interruptions, ...

cas x86 (ou IA32 - Pentium 32 bits) :

- 64 lignes de données (E/S),
- 32 lignes d'adresses
- quelques lignes de contrôle

OS : intermédiaire entre le matériel et les applications

→ L'OS permet l'exploitation du matériel [interface inférieure] offrant des services via une [interface supérieure] **unifiée** (plus pratique) aux applications

- il masque la complexité de l'implémentation et cache les contraintes physiques [Abstraction]
ex. taille mémoire limitée

→ *Gestionnaire des ressources*

matérielles : cpu, mémoire, périphériques, réseau, etc.

logicielles : applications, fichiers, etc

Objectifs :

- ① **Partage** : assurer une utilisation efficace et équitable,
ex. maximiser l'utilisation du CPU tout en répondant aux besoins des applications de façon équilibrée
- ② **Protection** : garantir l'intégrité, la stabilité et la sécurité du système face aux erreurs et abus.
ex. limitation des ressources (quotas, permissions).

Users

Applications

Operatin System (OS)

Hardware

Évolution des systèmes d'exploitation (1/2)

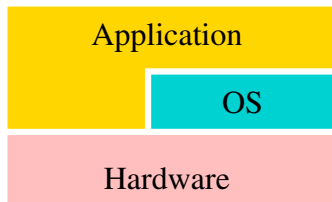
Une machine, un utilisateur, une application

- À l'origine, pas d'OS :
 - ▶ Premières machines : besoin d'un opérateur humain¹
 - ▶ Aujourd'hui : systèmes embarqués (voitures, ascenseurs)
- Entre temps : MS-DOS / mode non protégé

User (the)

Application (the)

Hardware



Systèmes monoprogrammés (monojob)

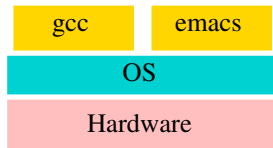
- **Problème** : usage inefficace des ressources matérielles : pas de recouvrement des E/S
- **Solution** : permettre la coexistence de plusieurs programmes dans le système afin de maximiser l'utilisation du matériel

¹Youtube video - How Do Operating Systems Work?

Évolution des systèmes d'exploitation (2/2)

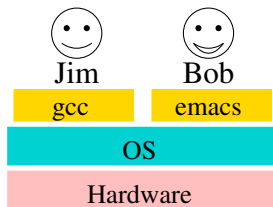
Systèmes multiprogrammés (multijob)

- Plusieurs programmes peuvent être chargés en mémoire en même temps.
Si un processus se bloque (E/S), un autre peut être exécuté
- Problèmes liés à la **concurrence** entre processus sont à résoudre : Si un processus fait une boucle infinie, écrit dans la mémoire d'un autre processus ?
→ **Fonction de protection** à assurer par l'OS



Systèmes multi-utilisateur (multiuser)

- apparus avec l'arrivée des terminaux (70's) :
mutualisation des ressources (1 machine/user : cher)
- Plusieurs utilisateurs sont autorisés à lancer leurs programmes sur la même machine.
- Problèmes liés à la **gestion** des utilisateurs sont à résoudre : utilisateur trop gourmand, mal intentionné, ...
→ **Fonction de protection** (OS)



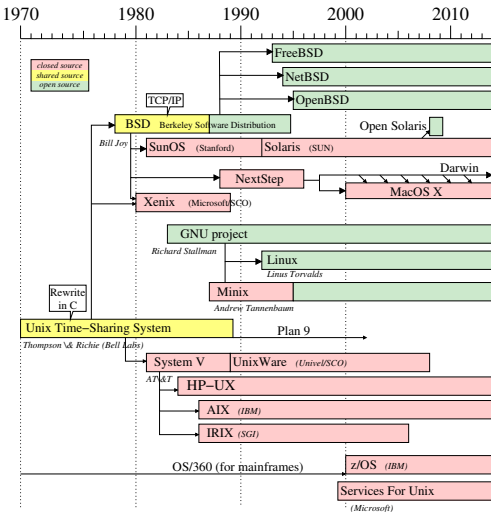
Pourquoi UNIX?

Qu'est ce qu'UNIX? Pourquoi étudier UNIX?

- Famille de systèmes d'exploitation, nombreuses implémentations
- Impact historique primordial, souvent copié
- Concepts de base simple, interface *historique* simple
- Des versions sont *libres*, tradition de code ouvert

Et pourquoi pas Windows?

- En temps limité, il faut bien choisir; je connais mieux les systèmes UNIX
- Même s'il y a de grosses différences, les concepts sont comparables
- D'autres cours à TELECOM Nancy utilisent Windows
occasion d'étudier ce système; volum horaire des cours systèmes est réduit
- Système plus *transparent* que Windows: plus facile de comprendre ce qui se passe sous le capot



Youtube Video - Histoire d'UNIX

1965 MULTICS: projet de système ambitieux (Bell Labs)

1969 Bell Labs se retire de MULTICS, Début de UNIX

1970 Unix projet Bell Labs officiel

1973 Réécriture en C
Distribution source aux universités
Commercialisation par AT&T

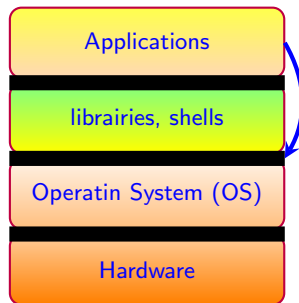
80-90 Unix War: BSD vs. System V

90-10 Effort de normalisation:

- ★ Posix1(88) processus, signaux, tubes
- ★ Posix1.b(93) semaphores, shared memory
- ★ Posix1.c (95) threads
- ★ Posix2 (92) interpréteur
- ★ Posix:2001,2004,2008,2016,202x Mises à jour

Organisation en couches et notion d'interface

- L'organisation en couches facilite la conception et permet la protection du matériel
- Un service (d'une couche à une autre) est caractérisé par son **interface** = l'ensemble des fonctions accessibles aux utilisateurs du service
- Chaque fonction est définie par la description de :
 - ▶ son mode d'utilisation : le format (**syntaxe**)
 - ▶ son effet : la spécification (**sémantique**)
- Les descriptions doivent être précises, complètes (y compris les cas d'erreur), non ambiguës.



Principe: séparation entre interface et réalisation

- Facilite la portabilité
 - ▶ transport d'une application utilisant le service sur une autre réalisation
 - ▶ passage d'un utilisateur sur une autre réalisation du système
- Les réalisations sont interchangeable facilement

Dans POSIX

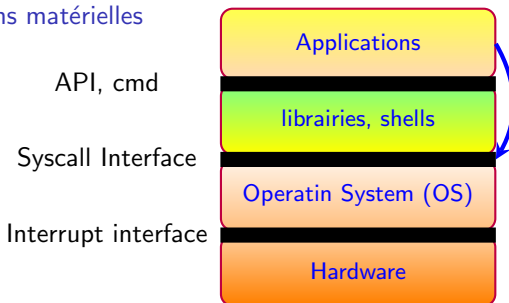
- C'est l'interface qui est normalisée, pas l'implémentation

Interfaces d'un OS type UNIX

Interface supérieure fournit des services aux processus utilisateurs

- Interface de programmation
 - ▶ Bibliothèques de fonctions C - **API** :
 - ▶ Les **appels systèmes** (fonctions spéciales)
- Interface de commande ou interface de l'utilisateur composée d'un ensemble de **commandes** :
 - ▶ textuelles - exemple : `rm *.o`
 - ▶ graphiques - exemple : déplacer l'icône d'un fichier

Interface inférieure composée d'un ensemble de sous-programmes invoqués pour le traitement des **interruptions matérielles**



Exemple d'usage des interfaces d'UNIX

Copier un fichier dans un autre

→ Appels systèmes

read et write offrent un contrôle fin sur les opérations d'E/S

```
while((nb_read = read(src, buf, sizeof(buf)))>0){
    nb_written = 0;
    while (nb_written < nb_read) {
        tmp = write(dest, buf + nb_written, nb_read - nb_written);
        if (tmp == -1) break; /* erreur */
        nb_written += tmp;
    }
}
```

→ Fonctions de librairie

fread et fwrite gèrent automatiquement les buffers et positions dans les fichiers

```
while((nb_read = fread(buf, 1, sizeof(buf), src))>0){
    if (fwrite(buf, 1, nb_read, dest) != nb_read) {
        break; /* erreur */
    }
}
```

→ Interface de commande shell: `$ cp src dest`

Documentation :

`man 1 <nom>`: documentation des commandes (par défaut)

`man 2 <nom>`: documentation des appels système

`man 3 <nom>`: documentation des fonctions

Résumé du chapitre

Yet anothe video !